

Implement any one of the following Expert System

```
# =====
```

```
# AIRLINE SCHEDULING & CARGO EXPERT SYSTEM
```

```
# =====
```

```
class AirlineExpertSystem:
```

```
    def __init__(self):
```

```
        # Knowledge Base (Rules)
```

```
        self.rules = [
```

```
            # Flight Scheduling Rules
```

```
            {
```

```
                "type": "flight",
```

```
                "conditions": {
```

```
                    "distance": "short",
```

```
                    "demand": "low"
```

```
                },
```

```
                "decision": "Assign small aircraft (ATR)"
```

```
            },
```

```
            {
```

```
                "type": "flight",
```

```
                "conditions": {
```

```
                    "distance": "long",
```

```
                    "demand": "high"
```

```
                },
```

```
    "decision": "Assign large aircraft (Boeing 777)"
  },
```

```
{
  "type": "flight",
  "conditions": {
    "weather": "bad"
  },
  "decision": "Delay or reschedule flight"
},
```

```
{
  "type": "flight",
  "conditions": {
    "maintenance": "required"
  },
  "decision": "Send aircraft for maintenance"
},
```

Cargo Scheduling Rules

```
{
  "type": "cargo",
  "conditions": {
    "cargo_type": "perishable"
  },
  "decision": "Use refrigerated transport"
},
```

```
{
```

```

        "type": "cargo",
        "conditions": {
            "cargo_type": "heavy"
        },
        "decision": "Assign cargo aircraft"
    },

    {
        "type": "cargo",
        "conditions": {
            "priority": "high"
        },
        "decision": "Use direct flight"
    },

    {
        "type": "cargo",
        "conditions": {
            "priority": "low"
        },
        "decision": "Use connecting flights"
    }
]

```

Inference Engine

```
def infer(self, facts, category):
```

```
    decisions = []
```

```
for rule in self.rules:
```

```
    if rule["type"] != category:
```

```
        continue
```

```
    match = True
```

```
    for key, value in rule["conditions"].items():
```

```
        if key in facts and facts[key] != value:
```

```
            match = False
```

```
            break
```

```
    if match:
```

```
        decisions.append(rule["decision"])
```

```
if decisions:
```

```
    return decisions
```

```
else:
```

```
    return ["No suitable decision found"]
```

```
# User Interface
```

```
def run(self):
```

```
    print("=====")
```

```
    print(" Airline Expert System ")
```

```
    print("=====")
```

```
    choice = input("Choose system (flight/cargo): ").lower()
```

```
facts = {}
```

```
# Flight System
```

```
if choice == "flight":
```

```
    facts["distance"] = input(  
        "Distance (short/long): ").lower()
```

```
    facts["demand"] = input(  
        "Demand (low/high): ").lower()
```

```
    facts["weather"] = input(  
        "Weather (good/bad): ").lower()
```

```
    maintenance = input(  
        "Maintenance required? (yes/no): ").lower()
```

```
    if maintenance == "yes":  
        facts["maintenance"] = "required"  
    else:  
        facts["maintenance"] = "not_required"
```

```
# Cargo System
```

```
elif choice == "cargo":
```

```
    facts["cargo_type"] = input(  
        "Cargo type (perishable/heavy/general): ").lower()
```

```
        facts["priority"] = input(
            "Priority (high/low): ").lower()

    else:
        print("Invalid choice")
        return

    # Get Decisions
    results = self.infer(facts, choice)

    # Display Results
    print("\nExpert System Decisions:")

    for result in results:
        print("-", result)

# Main Program
if __name__ == "__main__":

    system = AirlineExpertSystem()
    system.run()
```

Explanation of Airline Scheduling and Cargo Expert System

This program is a Rule-Based Expert System developed in Python. It helps in making airline flight scheduling and cargo management decisions using predefined rules. The system works like a human expert by checking conditions and giving suitable suggestions.

Main Components of Expert System

1. Knowledge Base

The knowledge base stores all rules of the system.

Each rule contains:

- Type → flight or cargo
- Conditions → input requirements
- Decision → action suggested by the system

Example:

```
{  
  "type": "flight",  
  "conditions": {  
    "distance": "long",  
    "demand": "high"  
  },  
  "decision": "Assign large aircraft (Boeing 777)"  
}
```

Meaning:

If flight distance is long and demand is high, then assign a large aircraft.

2. Inference Engine

The inference engine checks user inputs with all stored rules.

Function:

```
def infer(self, facts, category):
```

Working:

- Takes user input as facts
- Compares facts with rule conditions
- If conditions match, corresponding decision is added

- Finally displays all matching decisions
3. User Interface
The run() function interacts with the user.

It asks:

For Flight System:

- Distance
- Demand
- Weather
- Maintenance status

For Cargo System:

- Cargo type
- Priority

After taking input, it sends data to the inference engine and prints decisions.

Working of Program Step-by-Step

Step 1:

Program starts and displays:

Choose system (flight/cargo)

Step 2:

User selects either:

- Flight scheduling
OR
- Cargo scheduling

Step 3:

Program accepts required inputs.

Example:

Distance = long

Demand = high

Weather = good

Maintenance = no

Step 4:

Inference engine compares these inputs with rules.

Matched Rule:

distance = long

demand = high

Step 5:

System gives output:

Assign large aircraft (Boeing 777)

Flight Rules Explanation

1. Short distance + Low demand
→ Use small aircraft
2. Long distance + High demand
→ Use large aircraft
3. Bad weather
→ Delay or reschedule flight
4. Maintenance required
→ Send aircraft for maintenance

Cargo Rules Explanation

1. Perishable cargo
→ Use refrigerated transport
2. Heavy cargo
→ Use cargo aircraft
3. High priority cargo
→ Use direct flight
4. Low priority cargo
→ Use connecting flights to reduce cost

Advantages

- Fast decision making
- Reduces manual work
- Easy to modify rules
- Mimics expert-level reasoning

Applications

- Airline scheduling
- Cargo transportation

- Airport management systems
- Logistics and supply chain systems

Conclusion

This Airline Expert System uses rule-based logic to automate airline and cargo scheduling decisions. The system checks conditions entered by the user and provides suitable recommendations just like a human expert.